

Relatório de Integração — Sprint 2

1. Por que escolhi o RabbitMQ

Na descrição da Sprint estão como exemplo RabbitMQ e Redis Pub/Sub. Escolhi o RabbitMQ devido à persistência das mensagens. No Redis Pub/Sub, se o consumidor estiver offline quando a mensagem for publicada, ela simplesmente se perde. Isso seria um problema grave no SOS Reparos: imagina o encanador com o app fechado quando o cliente abre um chamado, ele nunca receberia a notificação. O RabbitMQ guarda as mensagens na fila até que alguém as consuma, o que resolve exatamente esse problema.

Outro ponto foi o painel de gerenciamento que vem junto com a imagem `rabbitmq:3-management`. Consegui visualizar as filas sendo criadas e as mensagens chegando em tempo real no navegador, o que ajudou muito para entender se a integração estava funcionando corretamente durante o desenvolvimento.

2. Como organizei os eventos

Optei por criar três filas separadas, cada uma com um propósito bem definido:

- **solicitacao.criada** — dispara quando um cliente abre um chamado
- **solicitacao.aceita** — dispara quando um prestador aceita
- **status.atualizado** — dispara em qualquer mudança de status

Poderia ter usado uma fila única com um campo indicando o tipo do evento, mas preferi separar porque cada app precisa escutar coisas diferentes. O prestador só precisa saber de novas solicitações; o cliente precisa saber quando o status muda. Separar as filas deixa isso mais claro e evita que cada consumidor precise filtrar mensagens que não são para ele.

3. Desafios que encontrei

O maior problema foi no começo: o servidor subia e tentava publicar antes do RabbitMQ estar pronto. Resolvi isso fazendo a conexão ser lazy, o publisher só conecta de verdade na primeira vez que tenta publicar algo, e depois reutiliza essa conexão. Simples, mas resolveu.

Também tomei a decisão de não deixar uma falha no RabbitMQ derrubar a API. Se o servidor não conseguir publicar um evento, ele loga o erro mas ainda responde ao cliente normalmente. O banco de dados já salvou a informação — o evento pode ser perdido, mas a operação principal não falha por causa disso.